

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Проект: AI Music Generator

1. Цель документа

Данное техническое задание определяет:

- функциональность приложения
- архитектуру системы
- зоны ответственности между участниками команды
- правила взаимодействия между клиентом и сервером

ТЗ используется как **основной ориентир при разработке** и может уточняться в ходе проекта.

2. Описание продукта

2.1 Краткое описание

AI Music Generator — мобильное приложение для генерации музыкальных треков с помощью искусственного интеллекта на основе текстового описания и параметров.

2.2 Основной сценарий пользователя

1. Пользователь просматривает онбординг приложения.
2. Пользователь регистрируется / входит в приложение (под вопросом создание аккаунтов, пока метим без создания аккаунта, то есть после онборда у нас сразу главный экран приложения)
3. Вводит описание желаемого трека
4. Выбирает параметры генерации
5. Запускает генерацию

- 6. Ожидает результат
- 7. Прослушивает и сохраняет трек

3. Роли в команде (для разработки)

Роль	Ответственность	Ответственный/ая
Flutter Lead	Архитектура мобильного клиента	Софа
Flutter UI / Design	Экраны, UX, плеер	Тася
Flutter API	Работа с backend, авторизация	Оля
Backend Lead	API-контракт, архитектура	Максим/Ростик
Backend Core	Пользователи, треки	Артем
Backend Worker	Очередь задач, AI-интеграция	Сашка
DevOps	Docker, окружение	Максим/Ростик
PM	Бэклог, тестирование, демо	Максим
QA	Тестирование	Валя

4. Функциональные требования

4.1 Пользовательские функции

4.1.1 Авторизация

- регистрация по email и паролю
- вход в систему
- хранение токена авторизации
- автоматический вход при повторном запуске

4.1.2 Генерация музыки

Пользователь может:

- ввести текстовое описание (prompt)
 - выбрать параметры:
 - жанр
 - настроение
 - темп
 - отправить запрос на генерацию
-

4.1.3 Статусы генерации

Каждая генерация имеет статус:

- **queued** — в очереди
 - **processing** — в процессе
 - **ready** — завершена
 - **failed** — ошибка
-

4.1.4 Работа с треками

Пользователь может:

- просматривать список сгенерированных треков
 - прослушивать трек
 - скачивать трек
 - удалять трек
 - добавлять в избранное
-

4.2 Системные функции

4.2.1 Очередь задач

- генерация выполняется асинхронно
 - задачи помещаются в очередь
 - отдельный worker обрабатывает задачи
-

4.2.2 Хранение данных

- метаданные хранятся в PostgreSQL
 - аудиофайлы хранятся в S3-совместимом хранилище
 - доступ к файлам осуществляется через временные ссылки
-

4.2.3 Developer Mode

- Возможность пользоваться приложением без ограничений
 - Придумать неоднозначный вход в него (ex. 25 раз кликнуть на версию приложения, открывается менюшка с паролем)
 - Функции для тестирования приложения на устройстве
-

5. Архитектура системы

5.1 Общая схема

- Flutter — клиентское приложение
- Go API — сервер приложений
- Go Worker — обработка генерации
- Redis — очередь задач
- PostgreSQL — база данных
- AI Music API — внешний сервис генерации

5.2 Взаимодействие

- клиент ↔ сервер: REST API (JSON)
 - генерация: асинхронная очередь
 - обновление статуса: polling (на первом этапе)
-

6. API (минимальный набор)

Auth

- `POST /auth/register`
- `POST /auth/login`

Jobs

- `POST /jobs`
- `GET /jobs/{id}`
- `GET /jobs`

Tracks

- `GET /tracks`
 - `GET /tracks/{id}`
 - `GET /tracks/{id}/download`
-

7. Технические требования

Клиент

- Flutter
- Android + iOS
- REST API
- хранение токена в secure storage

Сервер

- Go

- JWT авторизация
 - Docker
 - логирование ошибок
-

8. Репозиторий проекта

Используется **монорепозиторий**.

project/

```
├─ apps/
|   └─ mobile_flutter/
├─ services/
|   └─ api/
|   └─ worker/
├─ infra/
|   └─ docker-compose.yml
├─ docs/
|   └─ technical_spec.md
└─ README.md
```

9. Критерии готовности (Definition of Done)

Фича считается готовой, если:

- работает по сценарию
 - не ломает существующий функционал
 - задокументирована
 - протестирована вручную
-